

Python & POO

O que é programar, aplicações do Python e fundamentos da Orientação a Objetos



Prof.^a Waldinelly Costa



Curso DSM



Técnicas de Programação I



Fatec Indaiatuba



Agosto de 2026 | Aula 02

O que é programar?

A arte de ensinar uma máquina a resolver problemas

“

*"Programar não é sobre o que você sabe, mas
sobre o que você consegue descobrir."*

”

(ZELLE, 2016).

Na prática, programar é...

>_ Dar Instruções

Computadores não são inteligentes, eles são **extremamente obedientes** e rápidos.

Programar é fornecer um passo a passo exato (um algoritmo) na linguagem que a máquina entende para que ela execute uma tarefa.

💡 Resolver Problemas

É a arte de usar a **lógica**.

Pegamos um problema grande e complexo e o quebramos em partes pequenas e gerenciáveis. Em seguida, criamos soluções lógicas para cada uma dessas partes.

Python e suas aplicações

A linguagem de programação que dominou o mundo

Por que o Python é tão famoso?



Simples e Legível

Sua sintaxe (forma de escrever) parece um inglês simples. É considerada a melhor linguagem para iniciantes aprenderem lógica.



Muito Versátil

Você pode usar Python para criar sites, automatizar planilhas, desenvolver jogos e até mesmo programar robôs e inteligências artificiais.



Comunidade Gigante

Possui milhares de "bibliotecas" prontas. Alguém no mundo já resolveu o problema que você tem, e compartilhou o código de graça.

Onde o Python brilha?

```
def main():  
    ClearConsoleToColors(15, 1)  
    SetConsoleTitle("Project - Programming Code Abstract Screen")  
    int choice;  
    char ch = 'a', 'b';  
    while(1){  
        printf("1. Find Out the Day\n");  
        printf("2. Print all the day of month\n");  
        printf("3. Add Note\n");  
        printf("4. EXIT\n");  
        printf("ENTER YOUR CHOICE : ");  
        scanf("%d", &choice);  
        system("cls");  
        switch(choice){
```

Web (Back-End)

Criação da lógica de sites e sistemas usando frameworks como Django e Flask.



Ciência de Dados

Análise de milhares de dados, criação de gráficos e estatísticas com o Pandas.



Inteligência Artificial

Machine Learning, reconhecimento de voz e visão computacional (ex: ChatGPT).

Conceitos de POO

Programação Orientada a Objetos em Python

O Paradigma POO



Inspirado no Mundo Real

A POO é um "estilo" de programar. A ideia principal é tentar imitar o mundo real dentro do computador.

No mundo real, tudo é um **Objeto**. Um carro, um cliente, uma fatura. E todos os objetos possuem características (dados) e ações (comportamentos).



Organização Total

Em vez de ter um código solto e desorganizado com milhares de linhas, nós encapsulamos e organizamos o código em blocos lógicos.

Vantagem: O código fica muito mais fácil de ler, de corrigir bugs e, principalmente, de ser reaproveitado em projetos futuros.

1. Classes

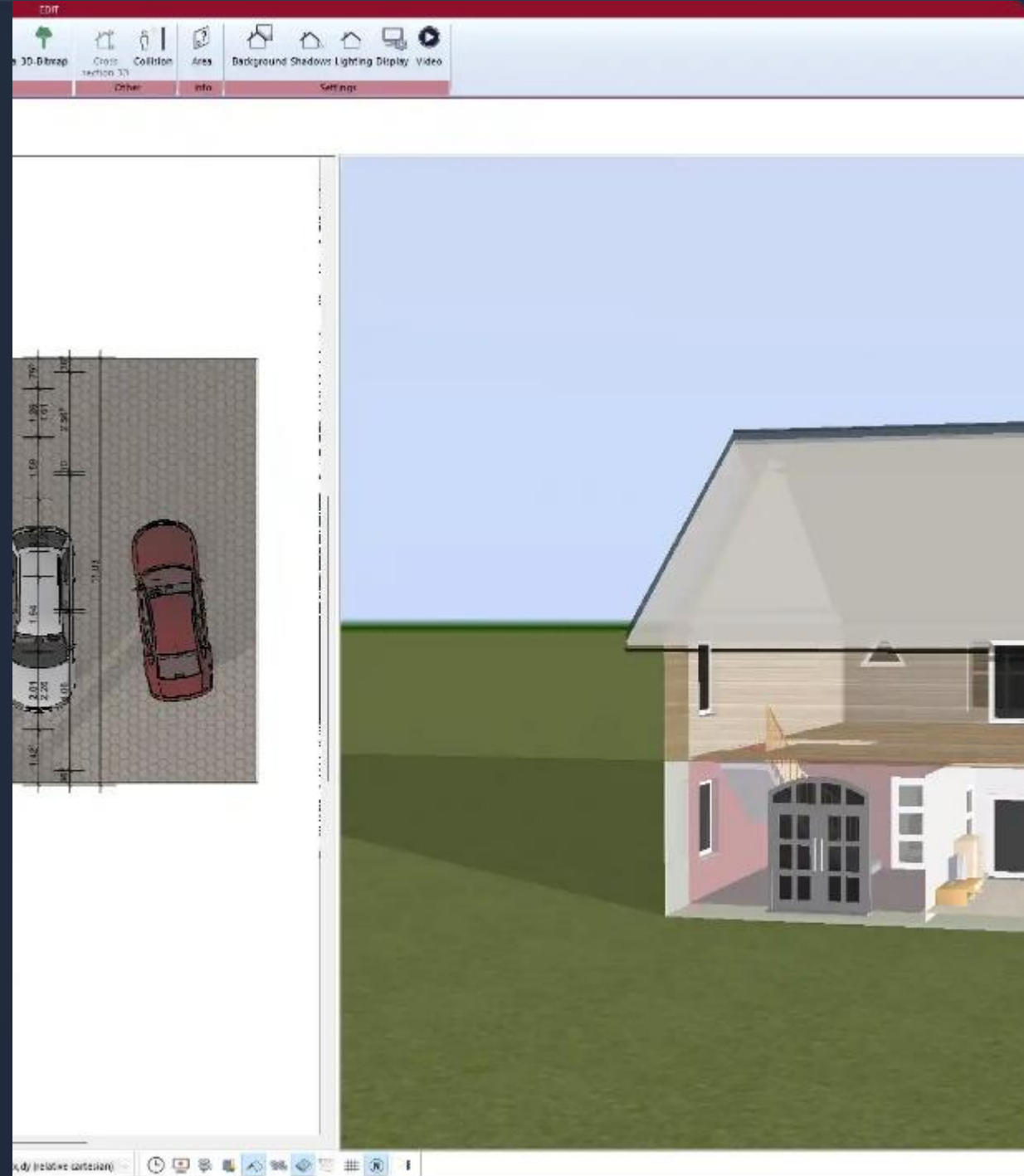
Conceito fundamental

O Molde (Blueprint)

Uma **Classe** é como se fosse a planta de uma casa ou a receita de um bolo. Ela não é o produto final, mas define as regras de como o produto será feito.

Exemplo Fácil: A planta de uma casa diz onde ficam as portas, a cor das paredes e qual o tamanho da sala, mas você não pode morar na planta de papel.

```
class Casa:
    def __init__(self, cor):
        self.cor = cor
```



2. Objetos

Conceito fundamental

O Produto Final

O **Objeto** é a casa real, de tijolos, construída a partir daquela planta (classe). Na programação, dizemos que o objeto é uma **instância** da classe.

Exemplo Fácil: A partir de uma única planta (Classe), podemos construir várias casas de verdade (Objetos). Uma pode ser pintada de azul, outra de amarelo.

```
minha_casa = Casa(cor="Azul")  
casa_do_vizinho = Casa(cor="Amarelo")
```



Classe vs. Objeto

A Classe

Definição: É o conceito abstrato, o molde, a definição do tipo de dado.

Exemplo Real: O conceito genérico de "Cachorro".
Sabemos que todo cachorro tem nome, raça e sabe latir.

Na Memória: Não ocupa espaço de dados no sistema.

O Objeto

Definição: É a entidade concreta, gerada e materializada a partir da classe.

Exemplo Real: O "Rex", que é especificamente um Poodle, tem 5 anos e late bem alto.

Na Memória: Ocupa espaço real (RAM), pois carrega os dados do "Rex".

3. Herança

Pilar da P00

Reaproveitando Características

Classes "filhas" podem herdar atributos e ações de uma classe "mãe", evitando que o programador tenha que reescrever códigos idênticos.

Exemplo Fácil: Temos uma classe mãe chamada **Veículo** (que tem rodas, motor e buzina). Criamos as classes filhas **Carro** e **Moto**. Ambas herdam as características do veículo, mas o Carro adiciona "Ar condicionado" e a Moto adiciona "Guidão".



4. Polimorfismo

Pilar da P00

Muitas Formas de Agir

Polimorfismo (várias formas) permite que objetos diferentes respondam ao **mesmo comando** de maneiras diferentes.

Exemplo Fácil: Imagine o comando **"Tocar"**. Se você der esse comando para um objeto "Violão", ele vibra as cordas. Se der para um objeto "Piano", ele bate martelos nas cordas. O comando no código é o mesmo, mas a execução muda de acordo com o objeto.



5. Encapsulamento

Pilar da P00

Protegendo os Dados

Serve para esconder como as coisas funcionam por dentro e proteger dados sensíveis para que não sejam alterados acidentalmente por outras partes do código.

Exemplo Fácil: Um **Cofre de Banco**. Você sabe que pode colocar e tirar dinheiro usando a senha certa. Porém, você não tem acesso (nem precisa entender) como o mecanismo interno das engrenagens da porta funciona.



Referência Bibliográfica

Leitura Recomendada

Para aprofundar os estudos em lógica de programação e dominar as estruturas da linguagem Python, recomendamos fortemente a leitura desta obra base do nosso curso:

ZELLE, John.

Python Programming: An Introduction to Computer Science.

3. ed. Franklin, Beedle & Associates, 2016.

